

Universidad Nacional de Río Cuarto
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Departamento de Computación
Licenciatura en Ciencias de la Computación

Bases de Datos II

Trabajo Práctico N.º 7 JSON y MongoDB

Alumno: Tomás Rodeghiero
Año: 2026

Resolución basada en el Teórico 9 (XML y JSON) y en experimentación con JsonPath/Gson en Java,
MongoDB y Compass.

Índice

1. Introducción	2
2. Ejercicio 1: Consultas JsonPath sobre el inventario	2
2.1. Marco teórico aplicado	2
2.2. Estructura del documento	3
2.3. Resolución por inciso	3
2.4. Ejecución (Java + Jayway)	4
3. Ejercicio 2: Generar un JSON único de facturas e items	4
3.1. Marco teórico aplicado	4
3.2. Esquema del documento generado	4
3.3. Estructura de la solución	5
3.4. Compilación y ejecución	5
4. Ejercicio 3: Diseño en MongoDB	5
4.1. Marco teórico aplicado	6
4.2. Esquema implementado	6
4.3. Script de creación (mongosh)	6
4.4. Ejecución	7
5. Ejercicio 4: Carga mínima	7
5.1. Datos de carga	7
5.2. Estrategia: <code>upsert</code> para idempotencia	8
5.3. Verificación	8
6. Ejercicio 5: Consultas MongoDB	8
6.1. Marco teórico aplicado	8
6.2. a) Listar nick, nombre, apellido y edad	9
6.3. b) Usuarios de más de 40 años	9
6.4. c) Noticias de autores de apellido Gonzalez, por fecha	9
6.5. Ejecución	10
7. Conclusiones	10

1. Introducción

El Práctico 7 introduce dos tecnologías centrales de los modelos documentales: **JSON** como formato semiestructurado y **MongoDB** como motor de base de datos orientado a documentos. Los cinco ejercicios cubren ambos lados:

- **Ejercicios 1 y 2:** trabajo con JSON desde Java. Consultas a un documento JSON con `JsonPath` (estilo `XPath` para JSON) y generación programática de un documento JSON a partir de una base relacional usando la biblioteca `Gson`.
- **Ejercicios 3 a 5:** modelado en MongoDB. Diseño de colecciones, carga de datos y consultas (incluyendo `$lookup` para joins entre colecciones).

Recursos del práctico. La cátedra provee un proyecto Java con `Gson` y `JsonPath`, copiado en `practico-07/recursos/json/`. Los documentos de ejemplo son `json.txt` (de la teoría) y `jsonInventario.txt` (del Ejercicio 1).

Verificación de las resoluciones existentes

Antes de redactar este documento se revisaron los cinco **README** y los scripts asociados. Conclusiones:

- **Ejercicio 1:** las expresiones `JsonPath` son correctas y consistentes con la sintaxis de `Jayway`. Se interpretó “bajo stock” como `cantidad < stock_min`, que es lo natural dado que `stock_min` está como campo raíz del JSON.
- **Ejercicio 2:** el programa Java con `JDBC` + `Gson` arma correctamente un único documento con facturas e ítems anidados.
- **Ejercicio 3:** el diseño con tres colecciones (`usuarios`, `noticias`, `comentarios`) refleja bien el E/R y respeta la cardinalidad de cada relación. Validadores `$jsonSchema` e índices únicos cargados.
- **Ejercicio 4:** la carga cumple el mínimo del enunciado (2 usuarios, 4 noticias, 2 comentarios por noticia) y es idempotente por `upsert`.
- **Ejercicio 5:** las tres consultas devuelven lo pedido; la (c) usa `$lookup` para resolver el join lógico entre `noticias` y `usuarios`.

No se encontraron errores. Lo que sigue documenta cada resolución explicando el porqué de las decisiones de modelado y la sintaxis usada.

2. Ejercicio 1: Consultas `JsonPath` sobre el inventario

Enunciado. Dado el documento `jsonInventario.txt`, resolver con `JsonPath`:

- (a) Todos los ítems del inventario.
- (b) Nombres de todos los ítems.
- (c) Ítems que están bajos en stock.
- (d) Colores de los ítems cuyo alto es mayor a 10 cm.

2.1. Marco teórico aplicado

JsonPath es un lenguaje de consulta para documentos JSON inspirado en `XPath`. Sus operadores clave:

Operador	Significado
\$	Raíz del documento.
@	Nodo actual (dentro de un filtro).
.	Acceso a hijo.
[*]	Comodín de elemento.
[?(...)]	Predicado/filtro.
..	Recorrido en profundidad.
[n] / [a:b]	Índice / rango de arreglo.

Operadores de filtro: ==, !=, <, <=, >, >=, =~ (regex), in, nin, size, etc.

2.2. Estructura del documento

```
{
  "inventario": {
    "items": [
      { "nombre": "tv", "cantidad": 25, "status": "A",
        "tamaño": { "alto": 14, "ancho": 21, "unidad": "cm" },
        "colores": ["negro", "amarillo"] },
      { "nombre": "notebook", "cantidad": 50, "status": "A",
        "tamaño": { "alto": 8.5, "ancho": 11, "unidad": "pulgadas" },
        "colores": ["rojo", "negro"] },
      { "nombre": "mesa", "cantidad": 100, "status": "D", ... },
      { "nombre": "planner", "cantidad": 75, "status": "D", ... },
      { "nombre": "postcard", "cantidad": 45, "status": "A",
        "tamaño": { "alto": 10, "ancho": 15.25, "unidad": "cm" },
        "colores": ["azul"] }
    ],
    "stock_min": 60
  }
}
```

Observaciones útiles para los filtros:

- stock_min está en la raíz, no dentro de cada ítem ⇒ se referencia con \$['stock_min'].
- La unidad de medida varía (cm/pulgadas) ⇒ para comparar el alto en cm hay que filtrar también por unidad.

2.3. Resolución por inciso

a) Todos los ítems

```
$ .inventario.items[*]
```

Devuelve el arreglo completo con los cinco objetos.

b) Nombres de todos los ítems

```
$ .inventario.items[*].nombre
```

Resultado:

```
["tv", "notebook", "mesa", "planner", "postcard"]
```

c) Ítems con bajo stock

Criterio: `cantidad < stock_min` donde `stock_min` está en la raíz.

```
$.inventario.items[?(@.cantidad < $('stock_min'))]
```

Cumplen la condición:

- tv (25)
- notebook (50)
- postcard (45)

d) Colores de ítems con alto > 10 cm

Filtramos por `unidad == "cm"` y `alto > 10`, y proyectamos los colores:

```
$.inventario.items[?(@.tamano.unidad == 'cm' && @.tamano.alto > 10)].colores[*]
```

Sólo tv (alto = 14, unidad = cm) cumple; postcard tiene alto exactamente 10. Resultado:

```
["negro", "amarillo"]
```

2.4. Ejecución (Java + Jayway)

La clase `practico-07/recursos/json/src/jsonPath/Ejercicio1Inventario.java`

arma estas consultas con la biblioteca de Jayway:

```
javac -cp "lib/*" -d bin src/jsonPath/Ejercicio1Inventario.java
java -cp "bin:lib/*" jsonPath.Ejercicio1Inventario
```

La salida puede traer una advertencia SLF4J inocua; los resultados de las cuatro consultas coinciden con lo que se calcula a mano arriba.

3. Ejercicio 2: Generar un JSON único de facturas e ítems

Enunciado. Hacer un programa Java (con la biblioteca Gson) que cree un único documento JSON con la información de la tabla `factura` y sus `items` de la base del Práctico 1.a.

3.1. Marco teórico aplicado

Gson es la biblioteca de Google para serializar Java a JSON. La idea del ejercicio es *aplanar* un modelo relacional con dos tablas (1:N entre `factura` e `item_factura`) a un único documento JSON con un arreglo de facturas y, dentro de cada factura, un arreglo anidado de ítems. Este patrón es el que MongoDB y otros motores documentales aprovechan para evitar joins en lecturas.

3.2. Esquema del documento generado

```
{
  "generado_en": "2026-03-28T12:34:56.123-03:00",
  "facturas": [
    {
      "nro_factura": 1001,
      "nro_cliente": 1,
      "fecha": "2024-08-01",
      "monto": 106000.0,

```

```
    "items": [
      { "cod_producto": 102, "cantidad": 2, "precio": 8000.0 },
      { "cod_producto": 103, "cantidad": 1, "precio": 90000.0 }
    ]
  },
  ...
]
}
```

3.3. Estructura de la solución

Archivos del proyecto Java:

- `recursos/json/src/jsonPath/Ejercicio2FacturaJson.java`: main del programa.
- `recursos/json/config/db.mysql.properties`: parámetros de conexión y nombres de tablas.
- `recursos/json/lib/gson-2.8.6.jar`, `mysql-connector-j-8.0.33.jar`: dependencias.

El programa:

1. lee la configuración del `.properties`;
2. abre una conexión JDBC a `practico1a_mysql`;
3. lee todas las facturas en un `Map<Long, Factura>`;
4. lee los items y los agrupa por `nro_factura` asociándolos a la factura correspondiente;
5. arma una raíz `{ generado_en, facturas[] }`;
6. serializa con `new GsonBuilder().setPrettyPrinting().create().toJson(raiz)`;
7. guarda el resultado en `salida/facturas_items.json`.

3.4. Compilación y ejecución

Desde `practico-07/recursos/json`:

```
javac -cp "lib/*" -d bin src/jsonPath/Ejercicio2FacturaJson.java
java -cp "bin:lib/*" jsonPath.Ejercicio2FacturaJson config/db.mysql.properties
```

El programa imprime la ruta del archivo generado y la cantidad de facturas exportadas. En la corrida de prueba documentada en el `README` (sobre una instancia sin filas en `factura`), exportó cero facturas y produjo un documento JSON consistente con un arreglo vacío, validando que el flujo (conexión, lectura, serialización, escritura) funciona correctamente.

Por qué Gson y no construir el JSON a mano. Gson maneja escape de strings, números, fechas y null de manera correcta. Construir JSON con `StringBuilder` es propenso a errores (escape de comillas, codificación, fechas) y se rompe en cuanto los datos contienen un signo " dentro de un campo de texto.

4. Ejercicio 3: Diseño en MongoDB

Enunciado. Crear una base en MongoDB con colecciones que permitan cargar datos según el siguiente E/R: `usuarios` (`nick`, `nyape`, `edad`, `dirección` compuesta por `calle` y `altura`, `telefonos`), `noticias` (`codigo`, `texto`, `fecha`) que los usuarios crean (1:N), y `comentarios` (`codigo`, `texto`) que los usuarios *hacen* (1:N) *para* las noticias (1:N) con palabras clave.

4.1. Marco teórico aplicado

MongoDB es una base de datos **documental**: el dato unitario es un documento JSON/BSON y los documentos se agrupan en **colecciones** (equivalentes lógicos de las tablas pero sin esquema rígido). Cada documento tiene un `_id` único.

Modelado documental. A diferencia del modelo relacional, en documentales:

- los *atributos compuestos* se modelan como *objetos embebidos* (la `direccion = {calle, altura}` no necesita una colección aparte);
- los *atributos multivaluados* se modelan como *arreglos* (los `telefonos` viven dentro del usuario);
- las *relaciones* se resuelven con *embedding* (anidar el dependiente dentro) o *referencing* (guardar una clave que apunta a otra colección). No hay constraints de *foreign key*.

Decisión de modelado.

- `direccion` y `telefonos` van *dentro* de `usuarios` (no son entidades propias, son atributos del usuario).
- `noticias` y `comentarios` se mantienen como colecciones *aparte* con referencias por clave de negocio, porque su volumen y su ciclo de vida son independientes (un usuario puede crear muchas noticias y muchas vidas de la app pueden agregar comentarios a una misma noticia).

4.2. Esquema implementado

Colección usuarios.

```
{
  nick:      "<clave de negocio, unica>",
  nyape:     "<nombre y apellido>",
  edad:      <int>,
  direccion: { calle: "<string>", altura: <int> },
  telefonos: [ "<tel1>", "<tel2>", ... ]
}
```

Colección noticias.

```
{
  codigo:    <int, unico>,
  texto:     "<string>",
  fecha:     <Date>,
  autor_nick: "<ref logica a usuarios.nick>"
}
```

Colección comentarios.

```
{
  codigo:    <int, unico>,
  texto:     "<string>",
  autor_nick: "<ref logica a usuarios.nick>",
  noticia_codigo: <int, ref logica a noticias.codigo>,
  pal_claves: [ "<kw1>", "<kw2>", ... ]
}
```

4.3. Script de creación (mongosh)

El script `mongosh/01_schema.js` crea la base, las tres colecciones con `$jsonSchema` para validar tipos, e índices únicos sobre las claves de negocio (`nick`, `codigo`) más índices auxiliares para las

consultas por relación (`noticias.autor_nick`, `comentarios.autor_nick`, `comentarios.noticia_codigo`).

```
use("practico7_er_noticias");
db.dropDatabase();

db.createCollection("usuarios", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["nick","nyape","edad","direccion","telefonos"],
    properties: {
      nick: { bsonType: "string", minLength: 1 },
      nyape: { bsonType: "string", minLength: 1 },
      edad: { bsonType: "int", minimum: 0 },
      direccion: { bsonType: "object",
        required: ["calle","altura"],
        properties: {
          calle: { bsonType: "string", minLength: 1 },
          altura: { bsonType: "int", minimum: 0 }
        },
      },
      telefonos: { bsonType: "array", minItems: 1,
        items: { bsonType: "string", minLength: 1 }
      }
    }
  }
});

// ... similar para noticias y comentarios

db.usuarios.createIndex ({ nick: 1 }, { unique: true });
db.noticias.createIndex ({ codigo: 1 }, { unique: true });
db.comentarios.createIndex({ codigo: 1 }, { unique: true });
db.noticias.createIndex ({ autor_nick: 1 });
db.comentarios.createIndex({ autor_nick: 1 });
db.comentarios.createIndex({ noticia_codigo: 1 });
```

Por qué no embeber los comentarios dentro de la noticia. Es una decisión clásica de modelado documental. Embeberlos es atractivo si la cardinalidad típica es baja y la consulta dominante es “noticia + sus comentarios”. Acá los comentarios son entidades con `codigo` propio, `autor_nick` y `pal_claves`, y crece de a uno por interacción de usuario. Mantenerlos en una colección separada evita documentos que crezcan sin tope y simplifica los índices secundarios sobre los comentarios.

4.4. Ejecución

Desde `practico-07/ejercicio-03`:

```
mongosh < mongosh/01_schema.js
mongosh < mongosh/02_seed_data.js
```

5. Ejercicio 4: Carga mínima

Enunciado. Agregar a la base al menos 2 usuarios, 4 noticias y 2 comentarios por cada noticia.

5.1. Datos de carga

Se insertan:

- 2 usuarios: `moises`, `elias`.

- 4 noticias con `codigo` ∈ {2001, 2002, 2003, 2004}.
- 8 comentarios: 2 por noticia, con `codigo` ∈ {7001, ..., 7008}.

5.2. Estrategia: upsert para idempotencia

Si se ejecuta el script más de una vez, las inserciones simples fallarían por duplicado de `nick` y `codigo` (claves únicas creadas en el Ejercicio 3). Para evitarlo se usa `updateOne(..., { upsert: true })`: si el documento existe se actualiza, si no se inserta.

```
db.usuarios.updateOne(
  { nick: "moises" },
  { $set: {
    nyape: "Moises Gonzalez",
    edad: NumberInt(45),
    direccion: { calle: "San Martin", altura: NumberInt(123) },
    telefonos: ["358-111111", "358-222222"]
  }},
  { upsert: true }
);
// ... mismo patron para elias y para las 4 noticias / 8 comentarios
```

5.3. Verificación

Conteos esperados después de la corrida:

```
db.usuarios.countDocuments({ nick: { $in: ["moises", "elias"] } })
// 2

db.noticias.countDocuments({ codigo: { $in: [2001, 2002, 2003, 2004] } })
// 4

db.comentarios.countDocuments({ noticia_codigo: { $in: [2001, 2002, 2003, 2004] } })
// 8
```

Para confirmar que efectivamente hay 2 comentarios *por* noticia:

```
db.comentarios.aggregate([
  { $match: { noticia_codigo: { $in: [2001, 2002, 2003, 2004] } } },
  { $group: { _id: "$noticia_codigo", total: { $sum: 1 } } },
  { $sort: { _id: 1 } }
]);
```

Debe devolver cuatro filas, todas con `total = 2`.

6. Ejercicio 5: Consultas MongoDB

Enunciado. Sobre la base creada, resolver:

- Listar `nick`, `nombre`, `apellido` y `edad` de todos los usuarios.
- Listar los usuarios de más de 40 años.
- Devolver las noticias que crearon los usuarios de apellido `Gonzalez` ordenadas por fecha.

6.1. Marco teórico aplicado

find vs aggregate. `find` permite filtrar (primer parámetro) y proyectar (segundo parámetro). El **aggregation framework** (`aggregate`) es un pipeline de etapas (`$match`, `$project`, `$lookup`,

`$unwind`, `$sort`, `$group`, etc.) más expresivo: permite transformaciones que `find` no soporta, como calcular campos derivados o hacer *joins* entre colecciones con `$lookup`.

Decisión de modelado relevante. En este modelo, `nyape` es un único campo concatenando nombre y apellido. Para “listar nombre y apellido por separado” (inciso a) hace falta descomponerlo, lo que se hace con `$split` y `$arrayElemAt`.

6.2. a) Listar nick, nombre, apellido y edad

```
db.usuarios.aggregate([
  {
    $project: {
      _id: 0,
      nick: 1,
      edad: 1,
      nombre: { $arrayElemAt: [{ $split: ["$nyape", " "] }, 0] },
      apellido: { $arrayElemAt: [{ $split: ["$nyape", " "] }, -1] }
    }
  }
]);
```

Cómo funciona. `$split` parte `nyape` por espacios y produce un arreglo: por ejemplo "Moises Gonzalez" → ["Moises", "Gonzalez"]. `$arrayElemAt` con índice 0 toma el primer token (nombre) y con -1 toma el último (apellido). Esta convención cubre nombres compuestos como "Juan Carlos Perez" tomando Juan como nombre y Perez como apellido.

6.3. b) Usuarios de más de 40 años

```
db.usuarios.find(
  { edad: { $gt: 40 } },
  { _id: 0, nick: 1, nyape: 1, edad: 1 }
);
```

Aquí basta con `find`: filtro por `$gt` y proyección explícita de los campos relevantes, suprimiendo el `_id`.

6.4. c) Noticias de autores de apellido Gonzalez, por fecha

Como `noticias.autor_nick` es una *referencia lógica* a `usuarios.nick`, hay que “unir” las dos colecciones con `$lookup`:

```
db.noticias.aggregate([
  {
    $lookup: {
      from: "usuarios",
      localField: "autor_nick",
      foreignField: "nick",
      as: "autor"
    }
  },
  { $unwind: "$autor" }, // $lookup devuelve array
  { $match: { "autor.nyape": /gonzalez$/i } }, // apellido termina en Gonzalez
  { $sort: { fecha: 1 } },
  {
    $project: {
      _id: 0,
      codigo: 1,

```

```
    texto: 1,  
    fecha: 1,  
    autor_nick: 1,  
    autor: "$autor.nyape"  
  }  
}  
});
```

Pipeline paso a paso.

1. **\$lookup**: por cada noticia, agrega el array **autor** con los usuarios cuyo **nick** coincide con **autor_nick** (en el modelo, a lo sumo uno).
2. **\$unwind**: desempaqueta el array para que cada noticia quede emparejada con un único documento de **autor**.
3. **\$match**: filtra por apellido usando expresión regular insensible a mayúsculas. `/gonzalez$/i` matchea cualquier **nyape** que termine en “gonzalez”.
4. **\$sort**: ordena por fecha ascendente.
5. **\$project**: arma el resultado con los campos pedidos.

6.5. Ejecución

Desde `practico-07/ejercicio-05`:

```
mongosh < mongosh/01_queries_ej5.js
```

7. Conclusiones

El práctico recorre tres capas del trabajo con datos semiestructurados:

1. **Consultas sobre JSON** (Ejercicio 1): `JsonPath` es a JSON lo que `XPath` es a XML. Ofrece operadores de navegación (`.`, `[*]`, `..`), filtrado (`[?(...)]`) y agregaciones (`length`, `min`, `max`, `avg`).
2. **Generación de JSON desde código** (Ejercicio 2): el caso típico es leer un modelo relacional y emitir un documento JSON *aplanado*. `Gson` resuelve la serialización; la lógica de agrupado (facturas con sus items) queda del lado de la aplicación.
3. **Modelado documental con MongoDB** (Ejercicios 3–5): las decisiones de *embedding* vs *referencing*, las claves de negocio y los índices únicos son las herramientas para representar un E/R sin perder integridad. El *aggregation framework* cubre la mayoría de las consultas no triviales, incluyendo *joins* lógicos con **\$lookup**.

La idea común detrás del práctico es: *el modelo documental no es “relacional sin reglas”*. Tiene sus propias buenas prácticas (*embedding* por composición, *referencing* por relaciones independientes, validación con `$jsonSchema`, índices) y sus propias herramientas de consulta (`JsonPath` para JSON aislado, pipeline de agregación para MongoDB).